Digital Image Fundamentals - Lab 2

Course: ARTI 403 – Image Processing
Department: Computer Engineering, College of Computer Science and Information Technology[cite: 1]

Procedural Steps: Sampling, Quantization, Arithmetic, and Logical Operations

In [ ]:
```
# Step 1: Import required libraries[cite: 1]
import cv2[cite: 1]
import numpy as np[cite: 1]
import matplotlib.pyplot as plt[cite: 1]
from PIL import Image[cite: 1]
```

In [ ]:
```
# Step 2: Define helper functions for sampling, quantization, and plottin
def sample_image(image, factor):
    """
    Downsamples the image by the given factor.
    """
    height, width = image.shape[:2][cite: 1]
    sampled_image = cv2.resize(image, (width // factor, height // factor)
    return sampled_image[cite: 1]

def quantize_image(image, levels):
    """
    Reduces the number of grayscale levels in the image.
    """
    quantized_image = np.floor(image / (256 // levels)) * (256 // levels)
    quantized_image = quantized_image.astype(np.uint8)[cite: 1]
    return quantized_image[cite: 1]

def plot_images(original, sampled, quantized):
    """
    Plots the original, sampled, and quantized images side by side.
    """
    plt.figure(figsize=(12, 4))[cite: 1]

    plt.subplot(1, 3, 1)[cite: 1]
    plt.imshow(original, cmap='gray')[cite: 1]
    plt.title('Original Image')[cite: 1]
    plt.axis('off')[cite: 1]

    plt.subplot(1, 3, 2)[cite: 1]
    plt.imshow(sampled, cmap='gray')[cite: 1]
    plt.title('Sampled Image')[cite: 1]
```

```
    plt.axis('off')[cite: 1]

    plt.subplot(1, 3, 3)[cite: 1]
    plt.imshow(quantized, cmap='gray')[cite: 1]
    plt.title('Quantized Image')[cite: 1]
    plt.axis('off')[cite: 1]
    plt.show()[cite: 1]
```

In [ ]:
```
# Step 3: Run baseline sampling and quantization[cite: 1]
image_path = '../images/lena_gray_256.tif'[cite: 1]
sampling_factor = 14[cite: 1]
quantization_levels = 9[cite: 1]

original_image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)[cite: 1]
if original_image is None:
    print(f"Error: Unable to load image at {image_path}")[cite: 1]
else:
    sampled_image = sample_image(original_image, sampling_factor)[cite: 1
    quantized_image = quantize_image(original_image, quantization_levels)
    plot_images(original_image, sampled_image, quantized_image)[cite: 1]
```

Task 1: Varying Sampling and Quantization Parameters

Objective: Experiment with different sampling factors and quantization levels to observe spatial degradation (pixelation) and gray-level degradation (false contouring)[cite: 1].

In [ ]:
```
# Test 1: Moderate Sampling & Quantization
sampled_mod = sample_image(original_image, factor=4)
quantized_mod = quantize_image(original_image, levels=16)
plot_images(original_image, sampled_mod, quantized_mod)

# Test 2: Severe Sampling & High Level Quantization
sampled_severe = sample_image(original_image, factor=16)
quantized_high = quantize_image(original_image, levels=2)
plot_images(original_image, sampled_severe, quantized_high)
```

Observation:

- Increasing the **sampling factor** reduces image resolution, causing pixelation or blockiness due to loss of spatial detail[cite: 1].
- Decreasing the **quantization levels** forces continuous intensity ranges into fewer discrete bands, resulting in visible false contouring artifacts[cite: 1].

Task 2: Image Operations

Performing subtraction, constant addition, set difference, symmetric difference, and intersection on gray-scale images[cite: 1].

In [ ]:
```
# Load and resize images
img1 = Image.open('../images/lena_gray_256.tif').convert('L')[cite: 1]
img2 = Image.open('../images/cameraman.tif').convert('L')[cite: 1]

resize = (400, 400)[cite: 1]
img1 = img1.resize(resize, Image.Resampling.LANCZOS)[cite: 1]
img2 = img2.resize(resize, Image.Resampling.LANCZOS)[cite: 1]

im1arr = np.asarray(img1, dtype=np.int16)
im2arr = np.asarray(img2, dtype=np.int16)
```

In [ ]:
```
# 1. Subtraction: Image 1 - Image 2
subtraction = np.clip(im1arr - im2arr, 0, 255).astype(np.uint8)[cite: 1]
subtraction_img = Image.fromarray(subtraction)

# Display Subtraction
plt.figure()
plt.imshow(subtraction_img, cmap='gray')
plt.title('Image Subtraction (Lena - Cameraman)')[cite: 1]
plt.axis('off')
plt.show()
```

In [ ]:
```
# 2. Addition with a constant value (175)
added_constant = np.clip(im1arr + 175, 0, 255).astype(np.uint8)[cite: 1]
added_constant_img = Image.fromarray(added_constant)

# Display Brightened Image
plt.figure()
plt.imshow(added_constant_img, cmap='gray')
plt.title('Image + Constant (175)')[cite: 1]
plt.axis('off')
plt.show()
```

In [ ]:
```
# Cast arrays to uint8 for bitwise operations
im1_u8 = im1arr.astype(np.uint8)
im2_u8 = im2arr.astype(np.uint8)

# 3. Set Difference: A - B = A AND (NOT B)
set_diff = cv2.bitwise_and(im1_u8, cv2.bitwise_not(im2_u8))[cite: 1]

# 4. Symmetric Difference: (A - B) UNION (B - A) or Bitwise XOR
sym_diff = cv2.bitwise_xor(im1_u8, im2_u8)[cite: 1]
```

```
# 5. Intersection: A AND B
intersection = cv2.bitwise_and(im1_u8, im2_u8)[cite: 1]
```

In [ ]:
```
# Plot Set and Logical Operations
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(set_diff, cmap='gray')
axes[0].set_title('Set Difference (A AND NOT B)')[cite: 1]
axes[0].axis('off')

axes[1].imshow(sym_diff, cmap='gray')
axes[1].set_title('Symmetric Difference (A XOR B)')[cite: 1]
axes[1].axis('off')

axes[2].imshow(intersection, cmap='gray')
axes[2].set_title('Intersection (A AND B)')[cite: 1]
axes[2].axis('off')

plt.tight_layout()
plt.show()
```